

Tuples in Python

1. What are tuples

Tuples documentation

Tuples are one of Python's *Built-in Types*

You find detailed the documentation here among other built-in types

`https:`

`//python.readthedocs.io/en/latest/library/index.html`

Then choose: Sequence Types - list, tuple, range

To reach: `https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range`

- Built-in Types
 - Truth Value Testing
 - Boolean Operations — and, or, not
 - Comparisons
 - Numeric Types — int, float, complex
 - Boolean Type - bool
 - Iterator Types
 - Sequence Types — list, tuple, range
 - Text and Binary Sequence Type Methods Summary
 - Text Sequence Type — str
 - Binary Sequence Types — bytes, bytearray, memoryview
 - Set Types — set, frozenset
 - Mapping Types — dict

2. Create and use a tuple

Create a tuple

```
1 cities = ("Stockholm", "Oslo", "London")
2 temperatures = (15.5, 14.3, 17.0)
3
4 print(type(cities))
5 print(type(temperatures))
```

```
<class 'tuple'>
```

```
<class 'tuple'>
```

Concatenating tuples

With '+' two tuples can be concatenated.

```
1 cities_1 = ("Stockholm", "Oslo", "London")
2 cities_2 = ("Berlin", "Paris")
3 cities = cities_1 + cities_2
4 print(cities)
```

('Stockholm', 'Oslo', 'London', 'Berlin', 'Paris')

A new tuple is created, and stored in the variable `cities`.

3. Indexing and slicing tuples

Indexing tuples

As other *sequences* (e.g., strings),
lists can be indexed.

Important to remember:

You always start counting at 0

```
1 cities = ("Stockholm", "Oslo", "London")  
2 print(cities[0])  
3 print(cities[2])
```

Stockholm

London

Slicing tuples

Tuples can also be sliced.

To get ['there', 'world'], you start with the index for 'there', i.e. 1, and end with the index **after** 'world', i.e. 3

(remember that indexing starts at 0)

```
1 tokenized_sentence = ("Hello", "there", "world", "?")
2 print(tokenized_sentence[1:3])
```

('there', 'world')

Slicing tuples

from beginning or towards the end

You can also start from the beginning or continue to the end

```
1 tokenized_sentence = ("Hello", "there", "world", "?")
2 # From the start
3 print(tokenized_sentence[:2])
4
5 # Towards the end
6 print(tokenized_sentence[1:])
```

```
('Hello', 'there')
('there', 'world', '?'])
```

4. Tuples are immutable

However, tuples are immutable

There is an important difference between a list and a tuple:
While lists are mutable, and therefore can be changed.

```
1 tokenized_sentence = ["Hello", "there", "world", "?"]
2 tokenized_sentence[2] = "!"
3 print(tokenized_sentence)
```

```
['Hello', 'there', '!', '?']
```

Tuples are **immutable** and therefore can **not** be changed.

```
1 tokenized_sentence = ("Hello", "there", "world", "?")
2 tokenized_sentence[2] = "!" # Does not work
3 print(tokenized_sentence)
```

TypeError: 'tuple' object does not support item assignment

Tuple example

```
1 def print_trip_details(trip):
2     print("Travelling from " + trip[0] + " to " + trip[1])
3     print("Time travelled: " + str(trip[2]) + "\n")
4
5 trip_one = ("Stockholm", "Oslo", 6)
6 trip_two = ("Stockholm", "Warsaw", 17)
7
8 print_trip_details(trip_one)
9 print_trip_details(trip_two)
```

Travelling from Stockholm to Oslo

Time travelled: 6

Travelling from Stockholm to Warsaw

Time travelled: 17

5. Tuples as elements in lists

Tuples as elements in lists

This list contains four elements, in the form of tuples. Each tuple contains three elements.

```
1 trips = [("Stockholm", "Oslo", 6),
2          ("Stockholm", "Warsaw", 17),
3          ("Oslo", "Sundsvall", 12),
4          ("Berlin", "Warsaw", 5)]
5
6 for trip in trips:
7     print(trip)
```

```
('Stockholm', 'Oslo', 6)
('Stockholm', 'Warsaw', 17)
('Oslo', 'Sundsvall', 12)
('Berlin', 'Warsaw', 5)
```

Tuples as elements in lists

```
1 trips = [("Stockholm", "Oslo", 6),
2          ("Stockholm", "Warsaw", 17),
3          ("Oslo", "Sundsvall", 12),
4          ("Berlin", "Warsaw", 5)]
5
6 last_trip = trips[-1]
7 print("Last trip: " + str(last_trip))
```

Last trip: ('Berlin', 'Warsaw', 5)

```
1 def print_trip_details(trip):
2     print("Travelling from " + trip[0] + " to " + trip[1])
3     print("Time travelled: " + str(trip[2]) + "\n")
4
5 trips = [("Stockholm", "Oslo", 6),
6          ("Stockholm", "Warsaw", 17),
7          ("Oslo", "Sundsvall", 12),
8          ("Berlin", "Warsaw", 5)]
9
10 for trip in trips:
11     print_trip_details(trip)
```

Travelling from Stockholm to Oslo
Time travelled: 6

Travelling from Stockholm to Warsaw
Time travelled: 17

Travelling from Oslo to Sundsvall
Time travelled: 12

Travelling from Berlin to Warsaw
Time travelled: 5

6. Lists as elements in lists
(i.e. leaving the tuple topic)

Lists as elements in lists

```
1 very_small_corpus = ["Hello", "there", "world", "?"],  
2                     ["The", "sun", "shines", "today", "!"],  
3                     ["Goodbye", "!"]]  
4  
5 for sentence in very_small_corpus:  
6     print(sentence)
```

```
['Hello', 'there', 'world', '?']  
['The', 'sun', 'shines', 'today', '!']  
['Goodbye', '!']
```

```
1 very_small_corpus = [["Hello", "there", "world", "?"],
2                       ["The", "sun", "shines", "today", "!"],
3                       ["Goodbye", "!"]]
4
5 for sentence in very_small_corpus:
6     print("-----")
7     print(sentence)
8     for word in sentence:
9         print(word)
```

```
['Hello', 'there', 'world', '?']
```

```
Hello
```

```
there
```

```
world
```

```
?
```

```
-----
```

```
['The', 'sun', 'shines', 'today', '!']
```

```
The
```

```
sun
```

```
shines
```

```
today
```

```
!
```

```
-----
```